

LAB # 2 Process

Objectives

- To create parent-child processes using fork() system call.
- To create a process chain and prevent child processes from becoming zombie using wait() system call.
- To create a process fan and prevent child processes from becoming zombies using a wait() system call.
- To create a process tree and prevent child processes from becoming zombie using wait() system call.
- To execute a new process from a process using an exec() system call.

Pre-Lab Tasks

1. Testing of gcc compiler

- i) type a source file with printf "helloworld" msg
- ii) compilation statement : `$gcc -o helloworld.out helloworld.c`

1. Display of command line arguments

Type the code below in the file cmdargs.c, and then compile and execute with arguments from the command line and write the output

```
int main(int argc, char* argv[])
{ int i;
for(i=0; i<argc; i++) printf("%s\
    n", argv[i]);
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task1$ ./arg1
Hello from command line args
./arg1 Hello from command line args
```

LAB # 2 Process

In-Lab Tasks

header file: *unistd.h*, *stdio.h*

System call: *fork()*, *getpid()*, *getppid()*, *wait()*

Task1: Compile and run the following program and study its behavior

a) Write the output of the child process

I am child: my process id is 8878 and my parent process id is 8877

b) Write the output of the parent process

I am parent: my process id is 8877 and my child process id is 8878 and my parent process id is 8489

c) Mention any unusual thing noticed in the output of child process

The output of the child process appear after the parent process every time

Reason:

Theoretically, the execution order of parent and child processes after *fork()* is unpredictable because they run concurrently. In practice, the parent usually prints first and the child second due to the small time taken to initialize the child process and scheduling by the OS.

LAB # 2 Process

```
#include <stdio.h>
#include <unistd.h>

int main()
{ int pid;
  pid = fork();
  if(pid > 0)

  printf("I am parent: my process id is %d and my child
  process id is %d\n", getpid(),pid);

  else if (pid == 0)
  printf("I am child: my process id is %d and my parent
  process id is %d\n",getpid(), getppid());
  else
  printf("ERROR in executing fork()");

  return 0;
}
```

LAB # 2 Process

Task2a: Compile and run the following code and write the output. From the output Also draw the process interconnected diagram to show process chain or fan or tree.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char* argv[])
{ int pid;
  int i;
  for(i=0; i<4; i++){
    pid = fork();
    if(pid > 0)
      break;
    else if(pid == 0)
      continue;
    else
      printf("ERROR: In fork()");
  } //end of for
  printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
} //end of main
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task3$ ./fork2
My process id is 10857 and my Parent process id is 8489
My process id is 10858 and my Parent process id is 10857
My process id is 10859 and my Parent process id is 10858
My process id is 10860 and my Parent process id is 10859
My process id is 10861 and my Parent process id is 10860
```

Process interconnected diagram:

```
08489 (bash) => root process
|
10857 (child at iteration 0) => process 1
|
10858 (child at iteration 1) => process 2
|
10859 (child at iteration 2) => process 3
|
10860 (child at iteration 3) => process 4
|
10861 (child at iteration 4) => process 5
```

LAB # 2 Process

Task 2b: Modify the code to get the number of process value from the command line argument. Compile and run the program with number of process value other than 4 and write the output.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// for atoi()
int main(int argc, char* argv[])
{
    int pid;
    int i;
    for(i=0; i<atoi(argv[1]); i++)
    {
        pid = fork();
        if(pid > 0)
            break;

        else if(pid == 0)
            continue;
        else
            printf("ERROR: In fork()");

    } //end of for

    printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
} //end of main
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task4$ ./fork2 2
My process id is 13442 and my Parent process id is 8489
My process id is 13444 and my Parent process id is 13443
My process id is 13443 and my Parent process id is 13442
```

LAB # 2 Process

Task 2c: Write the modified code to prevent the parent process from terminating before the child processes through wait() system call and also write the output

system call: wait()

```
#include <stdio.h>
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
#include <sys/wait.h> // for wait
```

```
int main(int argc, char* argv[])
```

```
{ int pid;
```

```
int i;
```

```
for (i = 0; i < atoi(argv[1]); i++) {
```

```
    pid = fork();
```

```
    if (pid > 0) {
```

```
        wait(NULL);
```

```
        break;
```

```
    }
```

```
    else if (pid == 0) {
```

```
        continue;
```

```
    }
```

```
    else {
```

```
        printf("ERROR: In fork()\n");
```

```
    }
```

```
    } // end of for
```

```
printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
```

```
//end of main
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task2c$ ./wait 3
```

```
My process id is 206957 and my Parent process id is 206956
```

```
My process id is 206956 and my Parent process id is 206955
```

```
My process id is 206955 and my Parent process id is 206954
```

```
My process id is 206954 and my Parent process id is 206510
```

Explanation:

Parent process is waiting for its child that's why the 1st process output is displayed at the end

(parent)

LAB # 2 Process

Task 3a: Compile and run the following code and write the output. From the output Also draw the process interconnected diagram to show process chain or fan or tree.

```
#include <stdio.h>
#include <unistd.h>

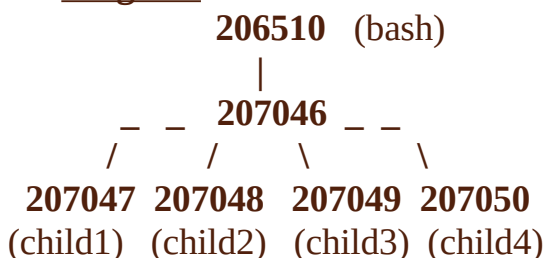
int main(int argc, char* argv[])
{ int pid;
  int i;
  for(i=0; i<4; i++){
    pid = fork();
    if(pid > 0)
      continue;
    else if(pid ==
      0)
      break;
    else
      printf("ERROR: In fork()");
  } //end of for

  printf("My process id is %d and my Parent process id is
  %d\n", getpid(), getppid());
} //end of main
```

Output:

```
My process id is 207047 and my Parent process id is 207046
My process id is 207048 and my Parent process id is 207046
My process id is 207046 and my Parent process id is 206510
My process id is 207049 and my Parent process id is 207046
My process id is 207050 and my Parent process id is 207046
```

Diagram



LAB # 2 Process

Task 3b: Write the modified code to get the number of process value from the command line argument. Compile and run the program with a number of process value other than 4 and write the output.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char* argv[]) { int pid;
int i;

for(i=0; i<atoi(args[1]); i++){
pid = fork();
if(pid > 0)
    continue;
else if(pid == 0)
    break;
else
    printf("ERROR: In fork()");
}//end of for

    printf("My process id is %d and my Parent process id is
    %d\n", getpid(), getppid());

}//end of main
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task3b$ ./3b 3
My process id is 9679 and my Parent process id is 9678
My process id is 9680 and my Parent process id is 9678
My process id is 9678 and my Parent process id is 9016
My process id is 9681 and my Parent process id is 9678
```

LAB # 2 Process

Task 3c: Write the modified code to prevent the parent process from terminating before the child processes through wait() system call and also write the output

system call: wait()

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char* argv[])
{ int pid;
  int i;
  for(i=0; i<atoi(argv[1]); i++){
    pid = fork();
    if(pid > 0){
      wait(NULL);
      continue;
    }
    else if(pid == 0)
      break;
    else
      printf("ERROR: In fork()");
  //end of for

  printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task3b$ ./3c 3
```

```
My process id is 9790 and my Parent process id is 9789
```

```
My process id is 9791 and my Parent process id is 9789
```

```
My process id is 9792 and my Parent process id is 9789
```

```
My process id is 9789 and my Parent process id is 9016
```

LAB # 2 Process

Task 4a: Creating the Process Tree, write the code and output similarly as in process chain and process fan with $n=4$ (total child count = 2^n)

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char* argv[]) {
    int pid;
    int i;
    for(i=0; i<4; i++){
        pid = fork();
        if(pid < 0){
            printf("ERROR: In fork()");
        }
        // both parent and child reproduce their childrens
    } //end of for

    printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
} //end of main
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task4$ ./tree
My process id is 5030 and my Parent process id is 5023
My process id is 5032 and my Parent process id is 5027
My process id is 5033 and my Parent process id is 5025
My process id is 5034 and my Parent process id is 5024
My process id is 5035 and my Parent process id is 5028
My process id is 5037 and my Parent process id is 5029
My process id is 5027 and my Parent process id is 5023
```

LAB # 2 Process

My process id is 5025 and my Parent process id is 5023
My process id is 5031 and my Parent process id is 5026
My process id is 5029 and my Parent process id is 5024
My process id is 5024 and my Parent process id is 5023
My process id is 5036 and my Parent process id is 5026
My process id is 5038 and my Parent process id is 5031
My process id is 5028 and my Parent process id is 5025
My process id is 5026 and my Parent process id is 5024
My process id is 5023 and my Parent process id is 4892

Tree:

parent (shell 4892)



LAB # 2 Process

Task4c: In Process Tree with n=4, Prevent the parent process from terminating before child processes and write the modified code and output

system call: wait()

hint: while(wait())>0);

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char* argv[])
{ int pid;
  int i;
  for(i=0; i<4; i++){
    pid = fork();
    if(pid < 0){
      printf("ERROR: In fork()");
    }
    // both parent and child reproduce their childrens
}//end of for

    // prevent parent from terminating before children
while (wait(NULL) > 0);

  printf("My process id is %d and my Parent process id is %d\n", getpid(), getppid());
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/lab2/tasks/task4$ ./tree wait
My process id is 5622 and my Parent process id is 5615
My process id is 5624 and my Parent process id is 5619
My process id is 5625 and my Parent process id is 5617
```

LAB # 2 Process

My process id is 5626 and my Parent process id is 5616
My process id is 5627 and my Parent process id is 5620
My process id is 5628 and my Parent process id is 5621
My process id is 5629 and my Parent process id is 5618
My process id is 5619 and my Parent process id is 5615
My process id is 5620 and my Parent process id is 5617
My process id is 5621 and my Parent process id is 5616
My process id is 5630 and my Parent process id is 5623
My process id is 5617 and my Parent process id is 5615
My process id is 5623 and my Parent process id is 5618
My process id is 5618 and my Parent process id is 5616
My process id is 5616 and my Parent process id is 5615
My process id is 5615 and my Parent process id is 4892

LAB # 2 Process

Task5: write the code to create a child process and execute ls -l command in the child process

system call: exec()

Because by design, exec() system calls **do not return on success**.*

If they return, it means something went wrong → so you can safely print an error message.

hint: explore the variants exec() call

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>    // for wait()
int main() {

    int pid = fork();
    if (pid < 0) {
        printf("ERROR: fork failed\n");
    }
    else if (pid == 0) {
        char *args[] = {"ls", "-l", NULL};
        execv("/home/ls", args);
        // If exec fails:
        printf("ERROR: exec failed\n");
    }
    else {
        wait(NULL);
        printf("Parent: Child process finished
execution.\n");
    }
    return 0;
}
```

```
muha-ahmad@latitude-7490:~/lab2/tasks/task5$ ./exec total 24
-rwxrwxr-x 1 muha-ahmad muhammad-ahmad 16136 Sep 18 13:32 exec
-rw-rw-r-- 1 muha-ahmad muhammad-ahmad 585 Sep 18 13:32 exec.c
-rw-rw-r-- 1 muha-ahmad muhammad-ahmad 342 Sep 11 11:07 wait.c
Parent: Child process finished execution.
```